Lesson 9 Teacher's Key: On the Scoreboard

Godot Engine - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

The push-based update-function pattern: one small function (update_hud()) is the single source of truth for what the display shows, called every time - and only when - the underlying data actually changes.

Level 1 - Full Worked Solution

- update_hud() lives on GameManager, alongside score, lives, enemies_remaining, on_enemy_destroyed(), and on_enemy_escaped() - the same dedicated manager node that's owned all of this game's shared state since Lesson 6. It reads score and lives and writes both Labels' text via get_node("../ScoreText")/get_node("../LivesText") - the same relative-path pattern already used to reach WinText and LoseText - and never changes score or lives itself, only reflects them.
- Every place that changes score or lives (currently on_enemy_destroyed() and on_enemy_escaped(), both on GameManager) calls update_hud() immediately afterward - plus one extra call in GameManager's own _ready() so the HUD is correct before anything has happened yet.
- This is a genuinely new structural idea this week: previous lessons separated different KINDS of game logic (hit vs. escape); this lesson separates game logic from DISPLAY - all still on the one manager node.

Highlighted lines are new this week; everything else is code you already have.

```
# GameManager's script (complete):
var enemies_remaining := 5
var lives := 3
var score := 0

func _ready():
    update_hud()

func update_hud():
    get_node("../ScoreText").text = "Score: " + str(score)
    get_node("../LivesText").text = "Lives: " + str(lives)

func on_enemy_destroyed():
    enemies_remaining -= 1
    score += 10
    update_hud()
    if enemies_remaining <= 0:
```

```
        get_node("../WinText").show()

func on_enemy_escaped():
    lives -= 1
    update_hud()
    if lives <= 0:
        get_node("../LoseText").show()
```

Why This Structure

- Centralizing all display-writing into one function means there's exactly one place to look if the HUD ever shows something wrong, and exactly one place to change if the HUD's format ever needs to change (e.g. adding a comma to large scores later).
- This pattern scales well: as more stats get added in later lessons (wave number, high score, etc.), they extend update_hud() rather than requiring new scattered Label-writing code - and they belong on GameManager too, the same one-shared-state-home-per-project principle Rule 13 established.

Level 2 / Level 3 Talking Points

- Ask students to predict what would go wrong if a function changed score directly without calling update_hud() - the number would be correct internally but wrong on screen, a good illustration of why the two need to travel together.
- Compare with Unity's approach this same lesson (a polling Update() instead of a push function) - ask which one this Godot pattern most resembles conceptually to Unity's own signal-based design from Lessons 6-8 (it's the same push philosophy, just applied to display instead of destruction).

Level 2 - complete code:

```
# Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
# Checks: 3 kills read exactly "Score: 30"; an escape moves lives but
never score;
# both numbers are correct the instant the game starts, not only after the
first kill or escape.
```

Level 3 - complete code (all goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
# GameManager's script - Level 3 (Theme Overrides for font size/color are
set on each Label in the Inspector):
func on_enemy_destroyed():
    enemies_remaining -= 1
    score += randi_range(5, 15)
    update_hud()
    if enemies_remaining <= 0:
        get_node("../WinText").show()
```

Common Student Mistakes

- Forgetting to call update_hud() inside GameManager's own _ready() - the HUD shows blank or default text until the first kill or escape.
- Changing score or lives directly inside update_hud() by mistake (it should only read them, never write them).

- Defining update_hud() or score on the main scene's script instead of GameManager's - breaks the one-dedicated-manager pattern and makes get_node("../ScoreText") resolve from the wrong place.
- Node name typos in the get_node() calls (e.g. "../ScoreLable" vs. the actual node name in the Scene tree) - case-sensitive and exact-match required.